Contrôleur de feux de circulation

Programmation structurée

1. Présentation du sujet

L'objectif est de réaliser un contrôleur de feux de circulation.



Ce dispositif, parfois appelé « automate spécialisé » de feux de circulation, prend l'aspect d'une armoire électrique à proximité d'un carrefour. Deux exemples sont illustrés ci-dessous.
Le contrôleur comporte une alimentation 24V, une CPU visible (carte à microcontrôleur ou boîtier spécialisé) et des relais pour commuter les courants des feux de signalisation.

Pour la CPU, vous utiliserez votre module Raspberry Pi Pico, que vous programmerez en CircuitPython, en respectant au mieux les conventions du langage ST, texte structuré.

Notre configuration matérielle est donc différente des automates que vous utilisez en TP d'automatisme, tout en présentant des ressemblances. L'alimentation 24V est semblable.
Un automate comporte bel et bien une CPU à microcontrôleur, mais celle-ci n'est pas visible. Elle est à l'intérieur d'un boîtier plastique résistant à un environnement industriel. Les modules d'entrées/sorties peuvent être dans le même boîtier ou sous forme de modules externes empilables. Enfin le logiciel de programmation est généralement propriétaire du fabricant de l'automate.





Planning
Ce sujet s'étalera sur huit séances. L'évaluation prendra la forme d'une présentation orale.
Vous testerez vos premiers codes sur plaque d'essai.
Puis vous fabriquerez votre carte et testerez les codes les plus complets sur celle-ci.
Enfin, lors des deux dernières séances, vous prendrez en main le logiciel d'automatisme Codesys.

Principe du carrefour

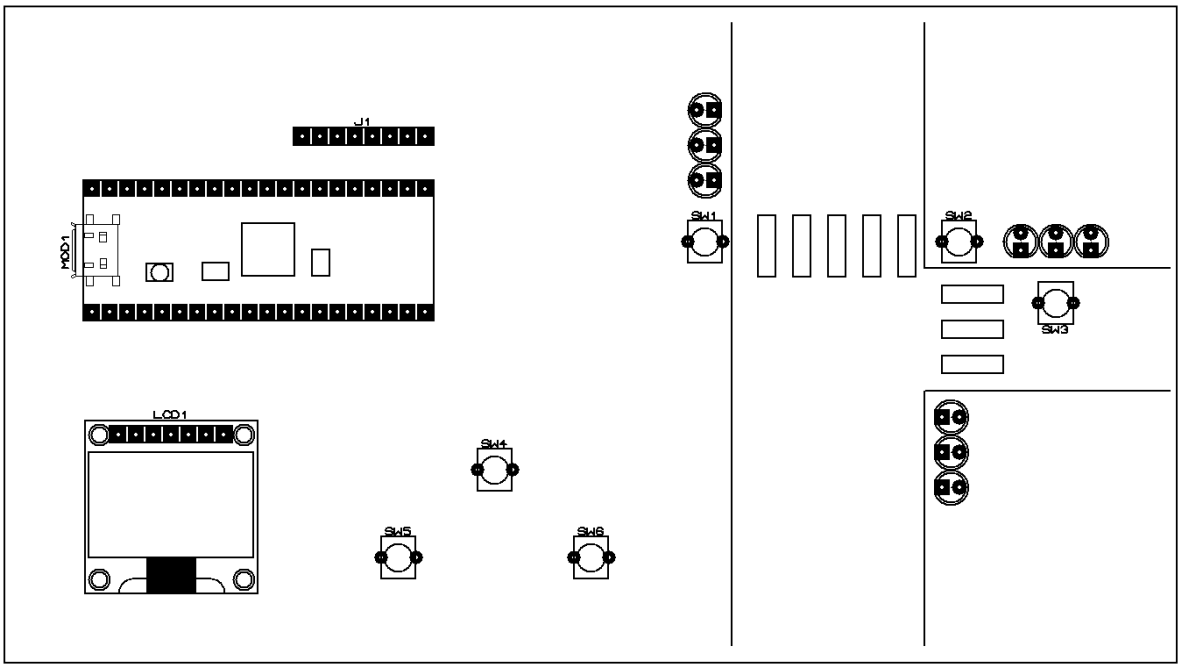
Ci-dessous le dessin de la carte que vous allez réaliser. A gauche le Raspberry Pi Pico, l'afficheur et trois boutons de contrôle, à droite les LEDs simulant le carrefour.

Il s'agit d'un carrefour en T. L'axe prioritaire, dessiné verticalement, est doté de deux feux tricolores, la rue secondaire à droite d'un seul.

SW1 et SW2 sont deux boutons poussoirs pour les piétons souhaitant traverser l'axe principal, et SW3 simule la boucle magnétique de détection d'un véhicule de la rue secondaire. Ces trois boutons poussoirs ont le même effet.

Généralement ce type de configuration fonctionne selon un des trois modes suivants :
- Mode cycle normal ou mode jour : les boutons sont sans effets, les feux font des cycles en boucle.
- Mode axe prioritaire ou mode nuit : le feu de l'axe principal est constamment vert. Un appui bouton, ou la détection d'un véhicule de l'axe secondaire, provoque un cycle unique des feux.
- Mode jour/nuit : mode cycle le jour et mode prioritaire la nuit.

J1 permet de relier votre carte à la maquette des feux.



2. Test des premiers codes sur plaque d'essai

Pour atteindre notre objectif, plusieurs difficultés sont à surmonter, comme respecter un codage structuré, créer une classe en Python. Vous allez donc adopter une méthode progressive.

Pensez à sauvegarder le code à chaque étape.
Sauvegardez votre travail sur votre disque personnel H:\ et sur un support externe.
Prévoyez un dossier pour ce projet et quelques sous-dossiers : CAO, Code, Enoncé, etc.

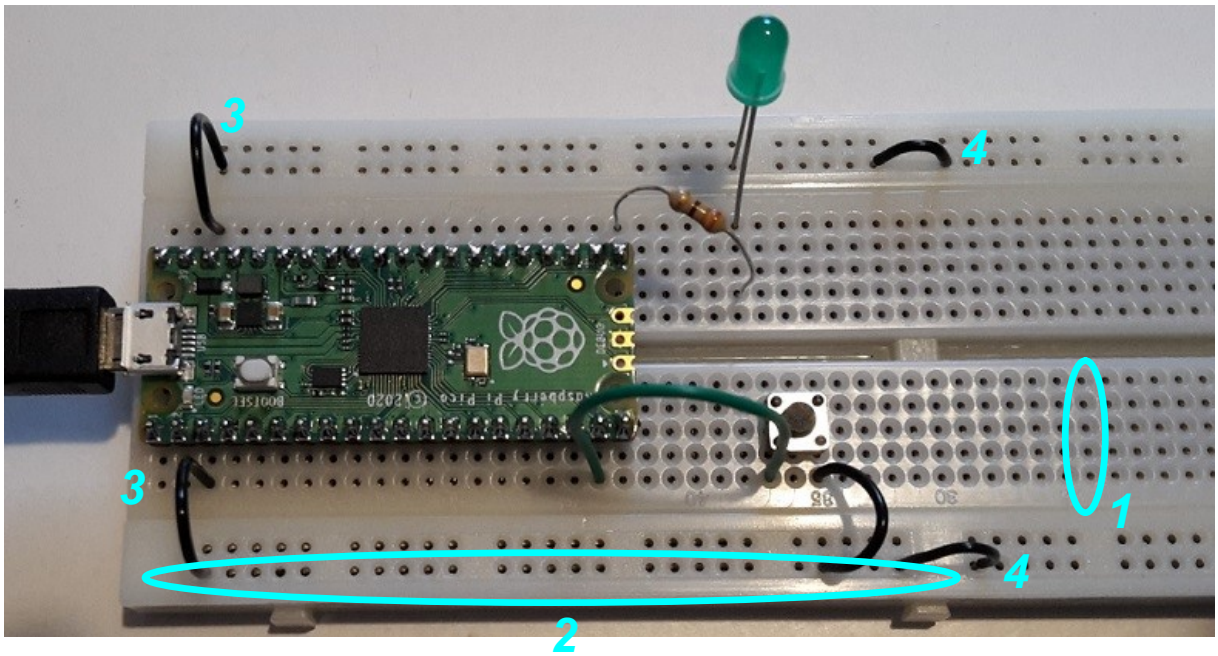
2.1 Une LED et un bouton poussoir - prise en main

Vous allez vous inspirer de la photo ci-dessous et du schéma complet donné un peu plus loin dans l'énoncé pour câbler une LED Feu1Vert correspondant au feu vert de la rue principale, ainsi que sa résistance R1, puis un bouton poussoir piéton BP_Pieton1.

Quelques détails sont mis en évidence sur l'image ci-dessous.
Repère 1 : la grande majorité des contacts de la plaque d'essai est reliée par colonne de cinq.
Repère 2 : il y a aussi des regroupements par ligne de vingt-cinq contacts, en haut et en bas de la plaque. On les réserve généralement pour les tensions d'alimentation.
Repère 3 : ces deux fils noirs permettent de créer des lignes GND depuis les broches 3 et 38 du Raspberry Pi Pico.
Repère 4 : ces deux fils noirs sont des ponts permettant, si nécessaire, de réunir deux lignes de 25 contacts en une seule ligne de 50 contacts.

N'oubliez pas la résistance de la LED. La patte la plus courte de la LED est la cathode, elle est reliée à la masse. Une des extrémités du bouton poussoir est reliée à la masse car l'entrée correspondante du Raspberry Pi Pico sera configurée avec une résistance de pull-up.

Faites vérifier votre câblage par un enseignant puis vous pourrez tester le code donné page suivante.



Remarquez que le code ci-dessous est composé de deux parties distinctes.

La première partie est exécutée une seule fois. Elle contient les importations des modules Python nécessaires et la configuration des entrées sorties. Plus tard elle contiendra la configuration des temporisations et l'initialisation de l'automate.

La seconde partie est une boucle, exécutée en permanence, elle contiendra le cycle de l'automate.

Pour comparaison, quand vous utilisez un logiciel dédié en TP d'automatisme, vous codez aussi ces deux parties. Vous commencez par définir vos entrées sorties à l'aide d'une interface dédiée. Puis vous écrivez votre cycle automate en langage à contact (langage ladder) ou en langage texte structuré (ST). Et même si vous ne vous en rendez pas compte parce que ça n'est pas explicite, votre bloc de code en langage à contact est bel et bien exécuté en boucle.

```
import board
from digitalio import DigitalInOut, Direction, Pull

# Configuration des entrées sorties
Feu1Vert = DigitalInOut(board.GP16)
Feu1Vert.direction = Direction.OUTPUT
BP_Pieton1= DigitalInOut(board.GP14)
BP_Pieton1.direction = Direction.INPUT
BP_Pieton1.pull = Pull.UP

while True :
    Feu1Vert.value = not BP_Pieton1.value
```

2.2 Trois LED et trois boutons poussoirs- programmation structurée

Ci-dessous un exemple simple de transcription d'un grafcet en programmation structurée. On passe d'une couleur de feu à la suivante en appuyant sur un bouton poussoir.

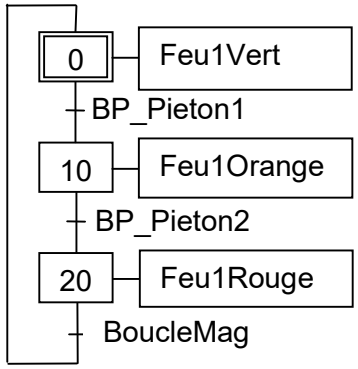
Vous câblerez sur plaque d'essai les trois LEDs d'un feu de l'axe principale ainsi que les deux boutons piéton et le bouton BoucleMag qui simule la boucle magnétique.

Vous configurerez les entrées sorties. Avant la boucle, vous ajouterez l'initialisation de l'automate à l'étape 0 selon l'exemple de code ci-dessous.

Dans la boucle infinie, le cycle automate est constitué de trois blocs distincts : conditions d'évolution, étapes et actions. Respectez ces notations. Le caractère très normé de ce style de codage lui procure clarté, robustesse et rapidité de développement. C'est assez facile à déboguer, à enrichir de nouvelles étapes.

D'aspect, on pourrait croire que le code est combinatoire. Mais remarquez dans le bloc des conditions d'évolution qu'une condition dépend toujours de l'étape antérieure. Le système est donc bien séquentiel.

Testez le fonctionnement du code.




```
# Initialisation de l'automate
X0 = 1
X10 = 0
X20 = 0

while True:

    # Les conditions d'évolution
    CE0_10 = not BP_Pieton1.value and X0
    CE10_20 = not BP_Pieton2.value and X10
    CE20_0 = not BoucleMag.value and X20

    # Les étapes
    if CE0_10 :
        X10=1
        X0=0
    if CE10_20 :
        X20=1
        X10=0
    if CE20_0 :
        X0=1
        X20=0

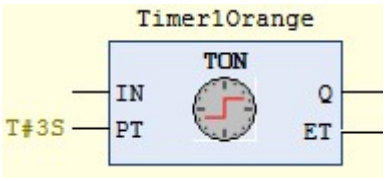
    # Les actions
    Feu1Vert.value = X0
    Feu1Orange.value = X10
    Feu1Rouge.value = X20
```

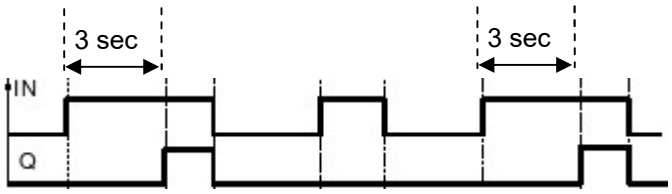
2.3 Une temporisation pour le feu orange

Il faut de nombreuses temporisations pour cadencer des feux de circulation.

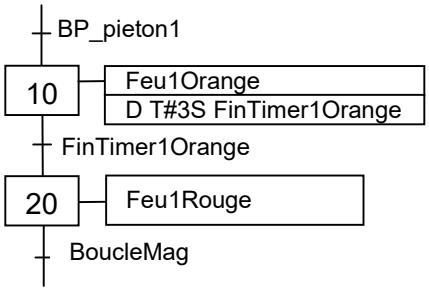
Vous allez commencer par remplacer le bouton BP_pieton2, qui permet page précédente de quitter le feu orange, par une temporisation, timer en anglais, de trois secondes.

Timer1Orange sera une temporisation à l'enclenchement. En automatisme on dit aussi de type TON.
Son fonctionnement est illustré ci-dessous.
Quand l'entrée IN passe à l'état haut, la sortie Q passe à l'état haut trois secondes plus tard.
L'entrée PT est le **preset time** et la sortie ET l'**elapse time**.





Le grafcet est modifié.
L'étape 10 active deux actions.
Le feu orange est activé immédiatement.
Une variable signalant la fin de la temporisation du feu orange est activée après un délai de trois secondes.
Cette variable est ensuite utilisée comme condition pour quitter l'étape 10.
Ainsi le feu orange est allumé pendant trois secondes.



Les logiciels dédiés à l'automatisme permettent de configurer ce type de temporisation.
Avec CircuitPython il va falloir la construire entièrement.
Vous allez donc quitter le rôle de l'automaticien pour endosser celui de l'informaticien industriel.

Dans une première étape, on vous propose de placer le bloc de code de la temporisation, donné ci-dessous, au début de la boucle. Il sera donc parcouru à chaque cycle automate.

```
while True:

    # Mettre à jour la sortie de la temporisation
    TimerPresetTime = 3
    if TimerIN :
        if TimerBusy == 0 :
            TimerStartTime = time.monotonic()
            TimerBusy = 1
        TimerElapsedTime = time.monotonic() - TimerStartTime
        if TimerElapsedTime >= TimerPresetTime:
            TimerQ=1
        else :
            TimerQ=0
    else :
        TimerQ = 0
        TimerBusy = 0
```

Le code est donné mais on attend de vous de l'avoir compris.

La fonction `time.monotonic()` donne le temps écoulé depuis l'allumage du Raspberry Pi Pico.
A condition d'importer le module en début de programme : `import time`

La condition d'évolution pour quitter l'étape 10 change :

```
CE10_20 = TimerQ and X10
```

Et dans les actions il faut ajouter la ligne :

```
TimerIN = X10
```

Il sera nécessaire d'initialiser certaines variables de la temporisation avant la boucle de l'automate. Laissez vous guider par les messages d'erreur de Mu Editor.

Terminez de modifier le code et testez la temporisation du feu orange.

2.4 Dupliquer les temporisations - utilisation d'une classe

Un feu simple nécessite trois temporisations, une pour chaque couleur de feu.
Il en faut six pour un carrefour complet.
Pour éviter de multiplier des lignes de code semblables vous allez utiliser une classe.



Une classe est un modèle à partir duquel on crée plusieurs objets semblables.
Vous pouvez procéder ainsi parce que Python est un langage de programmation orientée objet.

Un objet est un peu une super variable. Il est stocké en mémoire vive. Il comporte des attributs (variables appartenant à l'objet) et des méthodes (fonctions appartenant à l'objet).

Dans notre cas, les objets créés seront des temporisations.
Ils se nommeront `Timer1Vert, Timer1Orange, …`
L'attribut `Timer1Vert.IN` sera l'entrée de la temporisation du feu vert, `Timer1Vert.Q` sa sortie.
La méthode `Timer1Vert.maj()` mettra à jour la temporisation.

Ci-dessous le code de la classe **TON**, temporisation à l'enclenchement.
Remarquez que le code de la fonction **maj()** est très semblable au code de la page précédente.
Le préfixe **Timer** de la page précédente est ici remplacé par le préfixe **self**.
Lors de la création de l'objet de la temporisation du feu vert, le préfixe **self**. Sera remplacé par **`Timer1Vert.`**.
Ainsi chaque objet, ici une temporisation, possède ses attributs et méthode propres.

```
import time

class  TON :
    # Initialisation de la temporisation
    def __init__(self):
        self.IN = 0
        self.Busy = 0

    # Détermination de la valeur de la sortie de la temporisation
    def maj(self,duree) :
        self.PresetTime = duree
        if self.IN :
            if self.Busy == 0 :
                self.StartTime = time.monotonic()
                self.Busy = 1
            self.TimerElapsedTime = time.monotonic() - self.StartTime
            if self.TimerElapsedTime >= self.PresetTime :
                self.Q=1
            else :
                self.Q=0
        else :
            self.Q = 0
            self.Busy = 0
```

Le cahier des charges reste le même que dans la partie 2.3 : une temporisation uniquement pour le feu orange, mais en utilisant la classe **TON**.

Commencez par placer le code ci-dessus dans un fichier nommé **timer** et placé au même niveau que votre fichier **code**.

Depuis le fichier **code** pour accéder à la classe **TON**, il faut l'importer :

```
from timer import  TON
```

Avant la boucle de l'automate, vous créez, on dit instancier, votre objet temporisation du feu orange :

```
# Instanciation de la temporisation
Timer1Orange = TON()
```

L'initialisation des attributs de la temporisation se produit maintenant automatiquement lors de l'instanciation de l'objet. C'est le rôle page précédente de la première fonction `def __init__(self)`.

Pour la mise à jour de la temporisation en début de boucle, le bloc de code de l'exercice précédent est remplacé par l'appel à la méthode de mise à jour :

```
while True:

    # Mise à jour du timer
    Timer1Orange.maj(3)
```

La condition d'évolution pour quitter l'étape 10 change :

```
    CE10_20 = Timer1Orange.Q and X10
```

Et dans les actions, l'activation du timer change :

```
    Timer1Orange.IN = X10
```

Testez le code avec uniquement la temporisation du feu orange.

Puis quand cela fonctionne, remplacez les deux autres boutons par des temporisations.
Utilisez les durées suivantes pour les feux vert, orange et rouge : 5 secondes, 1 seconde, 8 secondes.
Au final vous devez avoir un feu de circulation tricolore unique, faisant des cycles.

Avant de poursuivre le codage d'un carrefour complet, vous allez fabriquer la carte.

3. Réalisation de la carte

3.1 Saisie du schéma

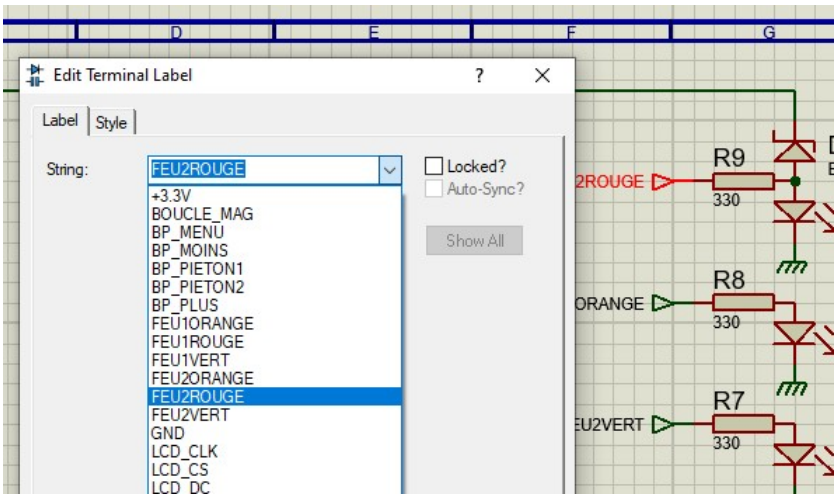
Le schéma de la carte est donné page suivante.

Ci-dessous le tableau des **devices**, c'est-à-dire des symboles des composants du schéma, et des **packages**, c'est-à-dire des boîtiers des composants.
Les composants sont traversants.

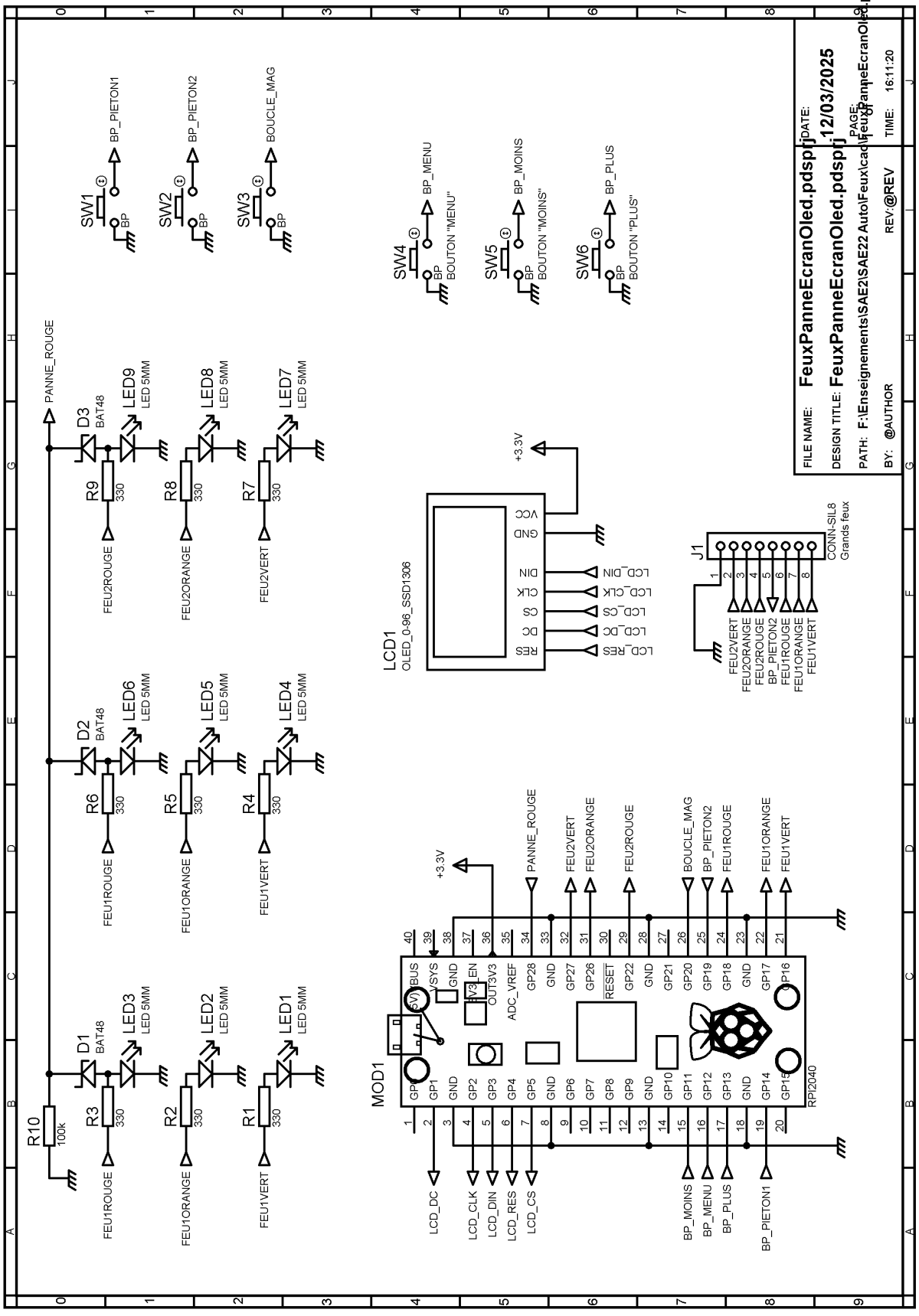
L'écartement entre broches du symbole du Raspberry Pi Pico est de 50TH. Donc dans le menu **Affichage** choisissez la grille d'accrochage **Snap 50th**.

DESIGNATION	DEVICES	PACKAGES
Raspberry Pi Pico	RPI2040	RPI2040
LCD OLED noir et blanc 0,96 inch	OLED_0-96_SSD1306	OLED_0-96_SSD1306_SPI
Connecteur header 8 points	CONN-SIL8	CONN-SIL8
LED 5mm	LED 5MM	LED 5MM
Diode Schottky	BAT48	DO35
Résistance	RES	RES40
Bouton poussoir SPST NO	BP	BP

Les erreurs concernant les noms des tensions sont trop fréquentes. Donc saisissez la première fois le nom d'un **Terminal Label**, puis utilisez la liste des tensions pour les labels portant le même nom suivants, comme illustré ci-dessous.



Les composants mal reliés sont aussi trop fréquents. Donc quand vous câblez, zoomez et faites des liaisons simples et courtes. Si vous doutez d'une liaison, déplacez le composant pour voir s'il est bien relié, et n'hésitez pas à effacer et recommencer une liaison.


FILE NAME: FeuxPanneEcranOled.pdsprj
DESIGN TITLE: FeuxPanneEcranOled.pdsprj
PATH: F:\Enseignements\SAE2\SAE22 Auto\Feux\cad\FeuxPanneEcranOled.pdsprj
BY: @AUTHOR
REV: @REV
DATE: 12/03/2025
TIME: 16:11:20
PAGE:
PANNE_ROUGE
R10 100k
D1 BAT48
D2 BAT48
D3 BAT48
R1 330
R2 330
R3 330
R4 330
R5 330
R6 330
R7 330
R8 330
R9 330
LED1 LED 5MM
LED2 LED 5MM
LED3 LED 5MM
LED4 LED 5MM
LED5 LED 5MM
LED6 LED 5MM
LED7 LED 5MM
LED8 LED 5MM
LED9 LED 5MM
FEU1VERT
FEU1ORANGE
FEU1ROUGE
FEU2VERT
FEU2ORANGE
FEU2ROUGE
SW1 BP_PIETON1
SW2 BP_PIETON2
SW3 BOUCLE_MAG
SW4 BP_MENU
SW5 BP_MOINS
SW6 BP_PLUS
BOUTON "MENU"
BOUTON "MOINS"
BOUTON "PLUS"
LCD1 OLED_0-96_SSD1306
MOD1 RPI2040
J1 CONN-SIL8 Grands feux
+3.3V
LCD_DC
LCD_CLK
LCD_DIN
LCD_RES
LCD_CS
BP_MOINS
BP_MENU
BP_PLUS
BP_PIETON1
BOUCLE_MAG

3.2 Placement des composants, tracé des pistes

Taille de la carte 170mm x 95 mm. On ne veut pas de cuivre trop proche du bord de la carte, donc tracez un second cadre à 50TH à l'intérieur du contour de la carte, par exemple en **Keep Out**.

Pour le placement, inspirez-vous de la page 2.
Attention au placement des LEDs. Ne confondez pas les LEDs de l'axe principal avec celles de l'axe secondaire.

Ne dessinez pas les rues et les passages piétons en cuivre mais au feutre.

Le circuit sera en simple face, donc faites en sorte de limiter les croisements de pistes. S'il y a un croisement, vous tracerez une piste côté Top courte et droite, mais que vous câblerez à l'aide d'un fil, ce qu'on appelle un strap.

Tracez des pistes de largeur 25TH et des vias de diamètre 80TH.

3.3 Câblage, rapide test avant d'insérer le module Raspberry Pi Pico

Récupérez les composants de la plaque d'essai pour les souder sur votre circuit imprimé.

Montez les LEDs rouges sur des supports tulipes pour qu'elles soient aisément démontables, pour simuler la panne d'un feu rouge.

Quand tous les composants sont soudés, au niveau du support du Raspberry Pi Pico, à l'aide de deux fils, vérifiez qu'il n'y a pas de court-circuit entre le +5V broche n°40 et la masse broche n°38.

4. Codage de la carte réalisée

4.1 Deux feux

Vérifiez que le code du feu tricolore du 2.4 que vous avez testé sur plaque d'essai fonctionne aussi sur la carte de circuit imprimé que vous venez de fabriquer.

Puis vous allez modifier le code pour commander les feux des deux rues.
Dessinez le graphcet correspondant. Il faut six temporisations.
Pendant une seconde les feux rouges des deux axes sont allumés en même temps
Utilisez les durées suivantes pour les feux vert et orange : 5 secondes et 1 seconde.
Vous pouvez prendre les mêmes durées pour les deux feux tricolores.

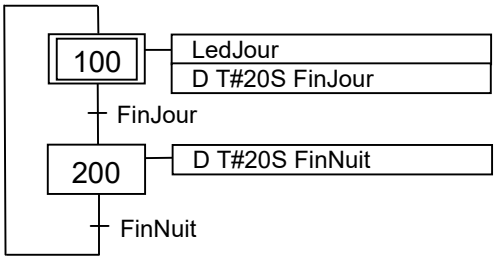
4.2 Mode jour/nuit

Ajoutez deux temporisations de 20 secondes pour simuler l'alternance du jour et de la nuit, selon le grafcet ci-dessous.

Utilisez la LED du Raspberry Pi Pico pour visualiser l'étape X100.
Elle peut être instanciée par l'instruction :

```
LedJour = DigitalInOut(board.LED)
```

Procédez par étapes.
Dans un premier temps faites simplement clignoter la LedJour.
Puis modifiez votre code afin que les feux fassent des cycles en boucle le jour et que la nuit le feu n°1 reste toujours au vert.
Enfin modifiez votre code pour qu'un appui sur un des trois boutons la nuit provoque un cycle unique des feux.



4.3 Test de l'afficheur pour la maintenance



Il faut qu'un technicien puisse régler les durées des feux à l'aide de trois boutons et d'un afficheur, placés dans le coffret métallique en bord de rue.

Depuis Moodle, copiez le dossier **lib.7z**. Dézippez le et placez son contenu dans le dossier **lib** de votre Raspberry Pi Pico. Il s'agit des librairies, nécessaires à l'affichage, extraites du paquet **adafruit_circuitpython_bundle**, disponible sur internet.

Ajoutez l'afficheur sur votre carte et testez le code ci-dessous, seul.

```
import board # n° des broches de la pico
import displayio # gestion d'afficheurs
import terminalio # gestion de la fonte
import busio # liaison série
from adafruit_display_text import label # gestion des zones de texte
import adafruit_displayio_ssd1306 # driver de l'afficheur
displayio.release_displays() # Libère les broches de l'afficheur
# Config du bus SPI de l'afficheur
spi = busio.SPI(clock=board.GP2,MOSI=board.GP3)
oled_reset = board.GP4
oled_cs = board.GP5
oled_dc = board.GP1
display_bus = displayio.FourWire(spi, command=oled_dc, chip_select=oled_cs,
reset=oled_reset, baudrate=1000000)
# Dimensions et instanciation de l'afficheur
WIDTH = 128
HEIGHT = 64
BORDER = 5
display = adafruit_displayio_ssd1306.SSD1306(display_bus, width=WIDTH,
height=HEIGHT)
# Make the display context
splash = displayio.Group()
display.root_group = splash
# Ajouter un label, une zone de texte
text_area = label.Label(terminalio.FONT, color=0xFFFFFF)
splash.append(text_area)
# Coordonnées et message du label
text_area.x = 20
text_area.y = BORDER+20
text_area.text = "Configuration\ndes feux"

while True:
    pass
```

Quand ce premier test de l'affichage fonctionnera, vous pourrez y associer vos deux fichiers des feux du 4.2 en externalisant l'affichage dans un fichier **maintenance**.
Vous aurez donc trois fichiers : **code** pour le cycle automate, **timer** pour la classe des temporisations, et **maintenance** pour le futur menu de la maintenance, dans lequel il n'y a pour l'instant que le test de l'afficheur.

Pensez à remplacez la boucle de l'affichage ci-dessus par une fonction, par exemple **config_feux()**, dans laquelle vous placerez pour l'instant les trois dernières lignes : coordonnées et message du label.

Puis appelez cette fonction depuis le fichier **code**, au tout début de la boucle du cycle automate. Ce sera le point d'accès au menu maintenance que vous allez créer, page suivante.

4.4 Menu maintenance

A chaque appui sur le bouton **menu** devra s'afficher en boucle :
- un écran vide ;
- le réglage de la durée de Timer1Vert,
- le réglage de la durée de Timer1Orange,
- …
- le réglage de la durée de Timer2Rouge,
- le choix du mode : jour, nuit, ou jour/nuit ;

les boutons + et – permettant le réglage des durées ou le choix du mode.



Dans votre fonction **config_feux()**, commencez par tester un bouton en vous inspirant de l'exemple ci-dessous.

Codez la fonction **afficher()** afin de simplement tester l'appui sur le bouton.
Vous y coderez plus tard l'affichage permettant le réglage des feux.

L'attente du relâchement du bouton évite de boucler dans le menu.
A terme, il sera préférable que le second **antirebond** soit commun à tous les boutons, car il ralentit le cycle de l'automate.

```
if not bp_menu.value : # appui du bouton bp_menu
    afficher()
    time.sleep(0.05) # anti rebond à l'appui
while not bp_menu.value : # attendre le relâchement du bouton
    pass
time.sleep(0.05) # anti rebond au relâchement
```

Puis codez le réglage du Timer1Vert à l'aide des boutons + et – et de l'afficheur.

La fonction **config_feux()** devra ensuite retourner la durée de Timer1Vert choisie.
Ainsi dans le fichier **code**, la durée de la temporisation pourra être mise à jour dans les premières lignes de code du cycle automate.

Ensuite enrichissez votre code pour permettre le réglage des six temporisations. Pour éviter de dupliquer des lignes de codes semblables, créez une liste des six durées. Modifiez cette liste avec les boutons, avant de la retourner à l'automate.

Enfin modifiez votre code pour inclure le choix du mode.

4.5 Détection de la panne

Pour des questions de sécurité, les feux rouges sont indispensables aux feux de circulation. Donc si l'un d'entre eux est défectueux, les feux orange clignotent.

Pour détecter la panne d'un feu rouge, vous allez mesurer la tension au niveau des anodes des LEDs rouges.

Sur le schéma D1 à D3 sont de type Schottky.
Que vaut la tension PANNE_ROUGE si les LEDs rouges fonctionnent correctement ? Que vaut cette tension si une des LEDs rouges est hors service (LED coupée, circuit ouvert) ?

La tension PANNE_ROUGE peut être mesurée par le Raspberry Pi Pico. En effet la broche GP28 est une des trois entrées du convertisseur analogique numérique.

Pour cela il faut importer le module du CAN, puis configurer l'entrée utilisée :

```
import analogio
panne_rouge = analogio.AnalogIn(board.A2)
```

Vous avez alors accès au résultat de la conversion à l'aide de l'attribut :

```
panne_rouge.value
```

Faîtes la mesure dans le fichier **maintenance**. Créez une variable **panne** que vous mettez à 1 quand une panne est détectée, et que vous mettez à 0 quand vous quittez le menu maintenance.
Ainsi la résolution de la panne nécessite l'intervention d'un technicien de maintenance, qui doit dérouler entièrement le menu.

4.6 Clignotement des feux orange

Les feux orange doivent clignoter :
- au démarrage des feux
- lors d'une panne d'un feu rouge

Ainsi la mise en route des feux, ou la panne d'un feu rouge, nécessite l'intervention d'un technicien de maintenance, qui doit dérouler le menu entièrement.

Le projet va comporter 4 grafcets.

Celui du cycle jour nuit, existant, est inchangé.

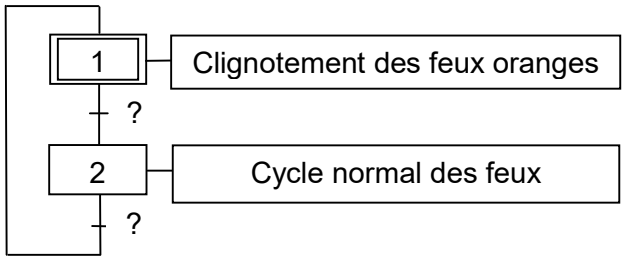
Celui du cycle des feux sera un peu modifié, selon l'exemple du second grafcet ci-dessous. On y retrouve les six étapes du cycle, étapes 20 à 70.

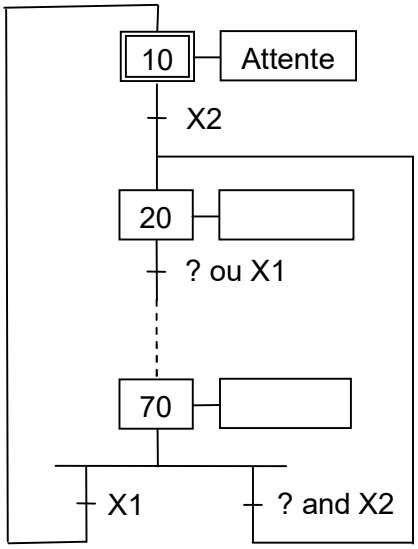
Celui du cycle de clignotement des feux orange devra être créé.

Enfin un grafcet superviseur doit être créé, selon l'exemple ci-dessous à terminer.
Il comporte deux étapes : **clignotement orange** et **cycle norma**l.

Les deux étapes du grafcet superviseur, X1 et X2, seront utilisées dans les conditions d'évolution des deux grafcets précédents pour qu'un seul des deux cycles soit exécuté : soit le clignotement orange soit le cycle normal des feux.

Un exemple de principe est donné dans le second grafcet ci-dessous, avec par la boucle de droite, le cycle normal des feux, et par la boucle de gauche l'accès à l'étape 10. Il s'agit d'une étape d'attente, sans action, pendant laquelle c'est le second grafcet, celui des feux clignotants (qui n'est pas donné ici), qui tourne. Et vice versa, quand le cycle normal des feux tourne, celui des feux clignotants orange est en attente.





Terminez le codage en texte structuré.